Lab 3: Spring Data JPA Basic

Persist the Todo app to MySQL database using Spring Data JPA

Open previous project from Lab 2.

What you will learn

- Add Spring Data JPA and MySQL Connector dependencies to pom.xml.
- Configure MySQL connection and Hibernate schema generation in application.properties.
- Annotate the Todo domain model with JPA annotations (@Entity, @Table, @Id, @GeneratedValue).
- Create TodoRepository by extending Spring Data JpaRepository.
- Refactor TodoService to use TodoRepository instead of the in-memory ArrayList.
- Verify that TodoController (Lab 1) and TodoRestController (Lab 2) require zero code changes.
- Confirm data persists across application restarts in both HTML and RESTful API.

Architecture Benefit: Zero Changes to Controllers

Because Lab 1 (Thymeleaf) and Lab 2 (REST API) both depend only on TodoService, swapping out the in-memory ArrayList for TodoRepository inside TodoService leaves both controller classes completely untouched.

Both HTML pages and RESTful endpoints immediately gain database persistence.

✓ Step 1. Add Spring Data JPA and MySQL to pom.xml

Open pom.xml and add two dependencies inside the <dependencies> block:

1. spring-boot-starter-data-jpa
2. mysql-connector-j

pom.xml (add inside <dependencies>)

```
<!-- Spring Data JPA: Hibernate + EntityManager + Spring Data -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- MySQL JDBC Driver -->
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
```

- spring-boot-starter-data-jpa: brings in Hibernate ORM, Jakarta Persistence API and Spring Data repositories.
- mysql-connector-j: official JDBC driver to communicate with MySQL Server.
- After saving pom.xml, reload Maven in your IDE.

✓ Step 2. Configure MySQL in the application.properties file

Open src/main/resources/application.properties and configure the JDBC connection and Hibernate behavior.

src/main/resources/application.properties

```
# MySQL DataSource configuration
spring.datasource.url=jdbc:mysql://localhost:3306/todolist_db?createDatabaseIfNotExist=true&us
eSSL=false&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=123456

# Hibernate JPA properties
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

- createDatabaseIfNotExist=true: MySQL driver automatically creates todolist_db if it does not already exist.
- spring.datasource.username / password: adjust to match your local MySQL credentials (default root/123456 or root/root).
- spring.jpa.hibernate.ddl-auto=update: Hibernate will create or alter tables automatically based on @Entity classes without dropping existing data.
- spring.jpa.show-sql=true: prints executed SQL statements to the console for debugging.

✓ Step 3. Project structure overview

Review your updated project layout.

We will

1. add repository/TodoRepository.java
2. update model/Todo.java and service/TodoService.java

Project package structure

```
src/main/java/com/example/todolist/
├── controller/
│   ├── TodoController.java
│   └── api/v1/
│       └── TodoRestController.java
├── dto/
│   └── TodoRequest.java
├── exception/
│   └── TodoNotFoundException.java
├── model/
│   └── Todo.java                    # updated with JPA @Entity annotations
├── repository/                      # data access interfaces
│   └── TodoRepository.java          # new JpaRepository interface
├── service/
│   └── TodoService.java             # refactored to use TodoRepository
└── TodolistApplication.java
```

✓ Step 4. Annotate Todo as a JPA Entity

Update model src/main/java/com/example/todolist/model/Todo.java.

We will add JPA annotations so Hibernate can map this class to a MySQL table.

src/main/java/com/example/todolist/model/Todo.java (Class and Fields)

```
package com.example.todolist.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import java.time.LocalDate;

@Entity
@Table(name = "todos")
public class Todo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String title;

    @Column(columnDefinition = "TEXT")
    private String description;

    @Column(name = "due_date")
    private LocalDate dueDate;

    @Column(nullable = false)
    private boolean completed;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }
    public LocalDate getDueDate() { return dueDate; }
    public void setDueDate(LocalDate dueDate) { this.dueDate = dueDate; }
    public boolean isCompleted() { return completed; }
    public void setCompleted(boolean completed) { this.completed = completed; }
```

Model Todo.java (continued)

```
    // Default no-args constructor required by JPA/Hibernate
    public Todo() {
    }

    public Todo(Long id, String title, String description, LocalDate dueDate, boolean
completed) {
        this.id = id;
        this.title = title;
        this.description = description;
        this.dueDate = dueDate;
        this.completed = completed;
    }

    public Todo(String title, String description, LocalDate dueDate, boolean completed) {
        this.title = title;
        this.description = description;
        this.dueDate = dueDate;
        this.completed = completed;
    }
 }
```

- @Entity: specifies that this class is an entity and mapped to a database table.
- @Table(name = "todos"): specifies the table name in MySQL.
- @Id and @GeneratedValue(strategy = GenerationType.IDENTITY): configures id as an auto-increment primary key in MySQL.
- public Todo(): JPA requires a no-argument constructor to instantiate objects when querying.

✓ Step 5. Create TodoRepository Interface

Create a new package repository and add TodoRepository.java. Extending JpaRepository provides ready-to-use CRUD operations.

src/main/java/com/example/todolist/repository/TodoRepository.java

```
package com.example.todolist.repository;

import com.example.todolist.model.Todo;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface TodoRepository extends JpaRepository<Todo, Long> {
}
```

- JpaRepository<Todo, Long>: first parameter is the Entity type (Todo), second is the primary key type (Long).
- Inherits built-in methods: findAll(), findById(id), save(entity), deleteById(id), count(), and more.
- You do NOT write an implementation class. Spring Data creates a proxy bean implementation automatically at startup.

✓ Step 6. Refactor TodoService to use TodoRepository

We wil refactor the src/main/java/com/example/todolist/service/TodoService.java to remove the in-memory ArrayList store and AtomicLong sequence, then delegate persistence to TodoRepository.

src/main/java/com/example/todolist/service/TodoService.java

```
package com.example.todolist.service;

import com.example.todolist.dto.TodoRequest;
import com.example.todolist.exception.TodoNotFoundException;
import com.example.todolist.model.Todo;
import com.example.todolist.repository.TodoRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
@Transactional
public class TodoService {

    private final TodoRepository todoRepository;

    public TodoService(TodoRepository todoRepository) {
        this.todoRepository = todoRepository;
    }

    @Transactional(readOnly = true)
    public List<Todo> findAll() {
        return todoRepository.findAll();
    }

    @Transactional(readOnly = true)
    public Todo findById(Long id) {
        return todoRepository.findById(id)
                .orElseThrow(() -> new TodoNotFoundException(id));
    }

    public Todo create(TodoRequest req) {
        Todo todo = new Todo(
                req.getTitle(),
                req.getDescription(),
                req.getDueDate(),
                req.isCompleted()
        );
        return todoRepository.save(todo);
    }

    public Todo update(Long id, TodoRequest req) {
        Todo todo = findById(id);
        todo.setTitle(req.getTitle());
        todo.setDescription(req.getDescription());
        todo.setDueDate(req.getDueDate());
        todo.setCompleted(req.isCompleted());
        return todoRepository.save(todo);
    }
```

```
    public void deleteById(Long id) {
        Todo todo = findById(id);
        todoRepository.delete(todo);
    }
}
```

- @Transactional: ensures methods execute inside a database transaction and roll back on uncaught runtime exceptions.
- readOnly = true: optimizes Hibernate performance for read-only queries by disabling dirty-checking.
- todoRepository.save(todo): inserts a new row if id is null, or updates the existing row if id is present.
- Notice that method names, parameters, and return types match Lab 1 and Lab 2 exactly, so neither controller needs changing.

✓ Step 7. Start Application and Verify Schema in MySQL

Run the application and verify that Hibernate automatically connects and creates the todos table.

1. Ensure your MySQL Server is running locally on port 3306.
2. Run TodolistApplication.java from your IDE.
3. Check console logs: look for Hibernate DDL output: create table todos (id bigint not null auto_increment, title varchar(100) not null, ... primary key (id)).
4. Open MySQL Workbench, HeidiSQL, or mysql CLI, and run: USE todolist_db; DESCRIBE todos; to inspect the generated schema.

✓ Step 8. Test Persistence with Browser, REST API and Server Restart

Execute this test sequence to verify that data is permanently stored in MySQL and shared between HTML and REST endpoints.

1. Open browser at http://localhost:8080/todos/new, fill title 'Finish Lab 3', and click Save.
2. Verify the new todo appears in the HTML table at http://localhost:8080/todos
3. Open http://localhost:8080/api/v1/todos: verify the exact same todo is returned in the JSON array.
4. Use Postman to send POST http://localhost:8080/api/v1/todos with JSON body: {"title": "Test JPA REST", "completed": false}. Response returns with generated id.
5. Check MySQL: SELECT * FROM todos; -> both rows are stored in the database.
6. STOP the Spring Boot server completely in your IDE.
7. RESTART the Spring Boot server.
8. Refresh http://localhost:8080/todos and http://localhost:8080/api/v1/todos: both todos remain present (no data loss!)
9. Send DELETE http://localhost:8080/api/v1/todos/1 in Postman: verify id 1 is deleted from MySQL and disappears from the HTML list.

✓ Step 9. Summary

- pom.xml includes spring-boot-starter-data-jpa and mysql-connector-j
- application.properties file correctly configures MySQL datasource and spring.jpa.hibernate.ddl-auto=update.
- Todo.java has @Entity, @Table, @Id, @GeneratedValue(strategy = IDENTITY), and a default no-args constructor.
- TodoRepository interface extends JpaRepository<Todo, Long>.
- TodoService uses TodoRepository with @Transactional and no longer uses ArrayList or AtomicLong.
- TodoController (Lab 1) and TodoRestController (Lab 2) remain completely unchanged.
- Data survives an application server restart and is visible in both MySQL and the web interfaces.